



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Избосарова Жасмина Олимжоновна

**РЕШЕНИЕ УРАВНЕНИЙ МАТЕМАТИЧЕСКОЙ ФИЗИКИ МЕТОДОМ PINN
И АНАЛИЗ УСТОЙЧИВОСТИ РЕШЕНИЯ МЕТОДОМ НЕЙРОННОГО
ТАНГЕНЦИАЛЬНОГО ЯДРА**

КУРСОВАЯ РАБОТА

Направление подготовки 02.03.01 сэт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

Студент гр. Б9123-02.03.01 сэт _____
(подпись)

Руководитель _____ К.С.Кузнецов
(подпись)

Оценка _____

Регистрационный № _____

(подпись) И.О. Фамилия

(подпись) И.О. Фамилия

«_____» _____ 2026 г.

«_____» _____ 2026 г.

г. Владивосток

2026

Введение

За последние полвека разработано значительное число алгоритмов аппроксимации и численного решения уравнений в частных производных. Среди современных подходов особое внимание привлекают методы, основанные на интеграции физических законов в архитектуру нейронных сетей, в частности метод Physics-Informed Neural Networks (PINN). Стремительный рост объёмов доступных данных и вычислительных ресурсов, а также достижения в области машинного обучения и анализа данных привели к существенным прорывам в различных научных дисциплинах, включая распознавание изображений [krizhevsky2012imagenet], когнитивные науки [alipanahi2015predicting] и геномику [3].

Вместе с тем при исследовании сложных физических, биологических и инженерных систем сбор экспериментальных данных зачастую сопряжён с высокими затратами, что приводит к необходимости работы в условиях ограниченной информации. В таком режиме традиционные методы машинного обучения, включая глубокие, сверточные и рекуррентные нейронные сети, как правило, демонстрируют недостаточную устойчивость и не обеспечивают строгих гарантий сходимости.

В данной работе рассматривается задача численного решения классического уравнения Курамото–Сивашинского с использованием метода Physics-Informed Neural Networks для аппроксимации и предсказания решения на основе известного аналитического выражения. Уравнение Курамото–Сивашинского относится к классу нелинейных эволюционных уравнений в частных производных и широко применяется для моделирования сложной пространственно-временной динамики, включая турбулентные режимы, распространение фронта пламени и динамику границ раздела фаз [добавить ссылку на литературу].

Несмотря на существование хорошо разработанных численных методов решения данного уравнения, таких как псевдоспектральные подходы и конечно-разностные схемы, оно характеризуется жёсткостью и склонностью к неустойчивости. Эти особенности обуславливают актуальность поиска альтернативных методов решения, в том числе основанных на использовании PINN, которые потенциально позволяют учитывать физические ограничения

задачи непосредственно в процессе обучения модели и повышать точность аппроксимации при ограниченном объёме данных.

Глава 1 Математическая постановка задачи и метод решения

1.1 Математическая модель и ключевые свойства

Работе рассматривается одномерная задача для уравнения Курамото–Сивашинского, являющегося нелинейным эволюционным уравнением в частных производных. В используемой постановке уравнение имеет следующий вид:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial^2 u}{\partial x^2} + \frac{\partial^4 u}{\partial x^4} = 0, \quad x \in [-L, L], \quad t \in [0, T]. \quad (1)$$

Данное уравнение описывает сложную пространственно-временную динамику и включает несколько физических механизмов. Нелинейный конвективный член $u \frac{\partial u}{\partial x}$ отвечает за перенос возмущений, диффузионный член второго порядка $\frac{\partial^2 u}{\partial x^2}$ характеризует процессы сглаживания, тогда как член четвёртого порядка $\frac{\partial^4 u}{\partial x^4}$ выполняет стабилизирующую функцию, подавляя рост высокочастотных возмущений.

Для рассматриваемой задачи известно точное аналитическое решение в виде бегущей волны:

$$u_{\text{exact}}(x, t) = b + \frac{15}{19} \sqrt{\frac{11}{19}} \left(-9 \tanh(k(x - bt - x_0)) + 11 \tanh^3(k(x - bt - x_0)) \right) \quad (2)$$

где параметры заданы следующим образом:

$$b = 5, \quad k = \frac{1}{2} \sqrt{\frac{11}{19}}, \quad x_0 = -12.$$

Данное решение получено путём редукции исходного уравнения в частных производных к обыкновенному дифференциальному уравнению с использованием замены переменной

$$\xi = x - bt, \quad (3)$$

что соответствует переходу к системе координат, движущейся со скоростью

(b), и позволяет представить решение в форме бегущей волны.

Начальные и граничные условия для рассматриваемой задачи задаются следующим образом:

$$u(0, x) = u_0(x), \quad (4)$$

$$u(t, -L) = u_1(t), \quad u(t, L) = u_2(t), \quad (5)$$

$$\frac{\partial u}{\partial x}(t, -L) = u_3(t), \quad \frac{\partial u}{\partial x}(t, L) = u_4(t). \quad (6)$$

Здесь первое выражение представляет собой *начальное условие*, определяющее распределение искомой функции ($u(x, t)$) в начальный момент времени ($t = 0$).

Второе и третье условия являются *граничными условиями Дирихле*, которые фиксируют значения функции на границах пространственной области ($x = -L$) и ($x = L$) для всех моментов времени.

Четвёртое и пятое условия представляют собой *граничные условия Неймана*, задающие значения пространственной производной ($\frac{\partial u}{\partial x}$) на границах области. Эти условия описывают поток или градиент величины (u) через границы и играют важную роль в обеспечении корректности постановки задачи, особенно для уравнений более высокого порядка.

Совместное задание начального условия и граничных условий Дирихле и Неймана формирует корректно поставленную краевую задачу для уравнения Курамото–Сивашинского и обеспечивает единственность и устойчивость её решения.

Выбор данной задачи обусловлен наличием точного решения, что делает её удобной для верификации численных методов. В частности, это позволяет количественно оценивать точность и устойчивость метода Physics-Informed Neural Networks (PINN), а также его способность воспроизводить нелинейную динамику одномерного уравнения Курамото–Сивашинского в условиях ограниченного объёма данных.

1.2 Описание метода Physics-Informed Neural Networks

Метод Physics-Informed Neural Networks (PINN) представляет собой подход, в котором нейронная сеть используется для аппроксимации решения дифференциального уравнения с явным учётом физических законов, задаваемых этим уравнением.

Обучение осуществляется посредством минимизации функционала ошибки, включающего вклад невязки дифференциального уравнения, а также отклонения от начальных и граничных условий. Невязка уравнения определяется путём подстановки аппроксимирующей функции в исходное дифференциальное уравнение, что позволяет учитывать его структуру непосредственно в процессе обучения. Таким образом, нейронная сеть обучается не только на данных, но и подчиняется физическим ограничениям задачи, что делает данный подход особенно эффективным в условиях ограниченного объёма наблюдений.

Реализация выполнена в среде Python с применением современных библиотек машинного обучения, обеспечивающих автоматическое дифференцирование и эффективную оптимизацию параметров модели.

Искомая функция аппроксимируется полносвязной нейронной сетью прямого распространения:

$$u_{\theta}(t, x) \approx u(t, x),$$

где θ — вектор параметров сети.

Входными параметрами сети являются пространственная и временная координаты, подвергнутые предварительному преобразованию, а выходом — значение искомой функции. Архитектура сети подбирается таким образом, чтобы обеспечить достаточную выразительную способность для аппроксимации нелинейного решения с выраженными пространственно-временными структурами.

Обучение модели осуществляется на совокупности точек рисунок 1, равномерно и случайным образом распределённых в области определения задачи. В частности, формируются точки, соответствующие начальному условию, граничным условиям и внутренней области.

Внутренние точки используются для минимизации невязки



Рисунок 1 – Обучающая выборка

дифференциального уравнения, получаемой путём подстановки нейросетевой аппроксимации в исходное уравнение. Производные по пространственной и временной переменным, включая производные высоких порядков, вычисляются с использованием автоматического дифференцирования.

Невязка уравнения имеет вид:

$$r_{\theta}(t, x) = \frac{\partial u_{\theta}}{\partial t} + u_{\theta} \frac{\partial u_{\theta}}{\partial x} + \frac{\partial^2 u_{\theta}}{\partial x^2} + \frac{\partial^4 u_{\theta}}{\partial x^4}.$$

Функция потерь формируется как сумма нескольких слагаемых:

$$\mathcal{L} = \lambda_r \mathcal{L}_r + \lambda_0 \mathcal{L}_0 + \lambda_b \mathcal{L}_b,$$

где

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} |r_{\theta}(t_i, x_i)|^2,$$

$$\mathcal{L}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |u_{\theta}(0, x_i) - u_0(x_i)|^2,$$

$$\mathcal{L}_b = \frac{1}{N_b} \sum_{i=1}^{N_b} (|u_{\theta}(t_i, -L) - u_1(t_i)|^2 + |u_{\theta}(t_i, L) - u_2(t_i)|^2).$$

Здесь N_r , N_0 , N_b — количество внутренних, начальных и граничных точек соответственно, а λ_r , λ_0 , λ_b — весовые коэффициенты.

Обучение сводится к задаче оптимизации:

$$\theta^* = \arg \min_{\theta} \mathcal{L}.$$

С целью повышения качества восстановления решения в работе применяется нормирование искомой функции:

$$u(t, x) = u_s \cdot \tilde{u}(t, x),$$

где u_s — масштабный коэффициент.

Кроме того, для повышения способности нейронной сети воспроизводить высокочастотные компоненты решения используется преобразование входных переменных с помощью Фурье-кодирования :

$$\gamma(t) = [\sin(2\pi B_t t), \cos(2\pi B_t t)], \quad \gamma(x) = [\sin(2\pi B_x x), \cos(2\pi B_x x)].$$

В ходе численного эксперимента проводится сравнение полученного нейросетевого решения с точным аналитическим решением. Это позволяет оценить точность метода, его устойчивость и способность корректно воспроизводить нелинейную динамику рассматриваемого уравнения. Анализ результатов показывает, что применение PINN в сочетании с нормированием переменных и использованием Fourier-признаков обеспечивает высокое качество аппроксимации и позволяет эффективно решать задачу даже при ограниченном объёме обучающих данных.

Глава 2 Этап 1. Численный эксперимент: генерация данных, архитектура сети и анализ эволюции решения

2.1 Формирование обучающего множества и коллокация

Вычислительный эксперимент для решения уравнения Курамото–Сивашинского (КС) проводился в прямоугольной пространственно-временной области $(t, x) \in [0, 4] \times [-30, 30]$. Суть метода PINN заключается в бессеточной аппроксимации: нейронная сеть $u_\theta(t, x)$ обучается минимизировать совокупный функционал невязки, опираясь на случайно распределенные облака точек, а не на фиксированную расчетную сетку рисунок 1.

Обучающая выборка была разделена на три функциональные группы:

1. *Точки начального условия (IC)*: Формируются при фиксированном времени $t = 0$ со случайным равномерным распределением координаты $x \in [-30, 30]$. В этих точках целевое значение $u(0, x)$ задается аналитическим профилем точного решения. Данный набор данных выступает в роли начального возмущения, закрепляя пространственную форму бегущей волны в стартовый момент времени.
2. *Точки граничных условий (BC)*: Генерируются на границах расчетной области $x = -30$ и $x = 30$ для случайных моментов времени $t \in [0, 4]$. Специфика уравнения КС, содержащего производную четвертого порядка u_{xxxx} , требует строгого контроля гладкости на границах. Поэтому функция потерь на границах (условия Дирихле и Неймана) стабилизирует поведение производных и предотвращает возникновение нефизичных осцилляций, проникающих внутрь области.
3. *Коллокационные точки (PDE residual)*: Плотное облако точек, равномерно сэмплированное внутри области $[0, 4] \times [-30, 30]$. В этих точках целевое значение функции неизвестно. Вместо этого с помощью механизма автоматического дифференцирования (AutoDiff) вычисляются

частные производные по входам сети и формируется физическая невязка:

$$r_\theta(t, x) = \frac{\partial u_\theta}{\partial t} + u_\theta \frac{\partial u_\theta}{\partial x} + \frac{\partial^2 u_\theta}{\partial x^2} + \frac{\partial^4 u_\theta}{\partial x^4}$$

Минимизация среднеквадратичного значения r_θ в коллокационных точках эквивалентна требованию, чтобы нейросетевая аппроксимация удовлетворяла физическим законам (балансу конвекции, антидиффузии и гипердиффузии), заложенным в уравнении.

2.2 Архитектурные особенности и борьба со спектральным смещением

Ключевой проблемой классических полносвязных нейронных сетей (MLP) при решении дифференциальных уравнений является так называемое «спектральное смещение» (spectral bias) — склонность сети в первую очередь аппроксимировать низкочастотные компоненты функции, игнорируя резкие градиенты. Для точного решения уравнения КС, имеющего вид суперпозиции функций $\tanh$ и $\tanh^3$, характерны выраженные локальные экстремумы (резкий пик и провал).

Для преодоления этого ограничения в архитектуру сети был внедрен слой Фурье-преобразования признаков (Fourier Feature Mapping). Входные координаты (t, x) перед подачей в скрытые слои отображаются в высокоразмерное пространство гармонических функций:

$$\gamma(v) = [\cos(2\pi Bv), \sin(2\pi Bv)]^T, \quad v \in \{t, x\}$$

Это преобразование искусственно обогащает спектр входного сигнала, позволяя модели эффективно захватывать высокочастотные пространственные детали фронта волны и корректно передавать их динамику во времени.

2.3 Анализ динамики обучения и эволюция аппроксимации

Анализ графиков сходимости (компонентов функции потерь и общей ошибки) позволяет глубоко понять нелинейную динамику обучения PINN. Обучение уравнения КС является жесткой задачей оптимизации. Слагаемые функции потерь (невязка PDE, начальные и граничные условия) имеют разную

чувствительность. В частности, член $\frac{\partial^4 u_\theta}{\partial x^4}$ вносит огромный вклад в градиенты на ранних этапах, что может приводить к дисбалансу оптимизации.

Эволюцию нейросетевого решения можно разделить на три характерные фазы:

Первая фаза - инициализации и теплового дрейфа (Начальные эпохи, рисунок 2):

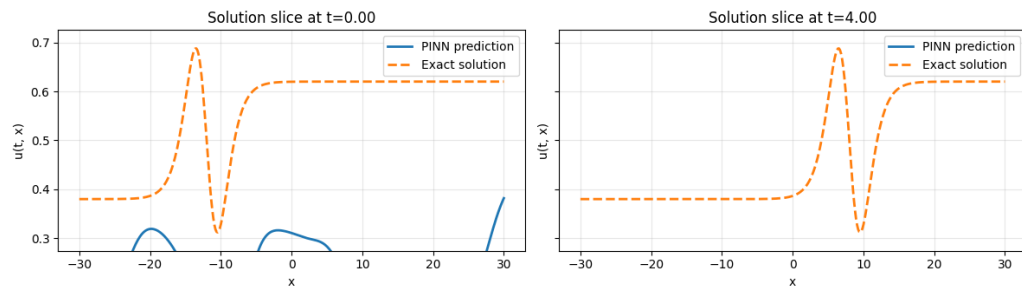


Рисунок 2 – Начальная эпоха

На старте обучения сеть инициализирована случайными весами. Предсказание (синяя линия) представляет собой константу или функцию с минимальной амплитудой, которая совершенно не совпадает с точным решением (оранжевая пунктирная линия) ни при $t = 0$, ни при $t = 4$. Модель пытается минимизировать общую ошибку тривиальным путем, сглаживая градиенты, так как штраф за неверные старшие производные u_{xxxx} на этом этапе подавляет штраф за несовпадение амплитуд.

Вторая фаза - структурное выравнивание (Промежуточные эпохи, рисунок 4, ??, рисунок 5):

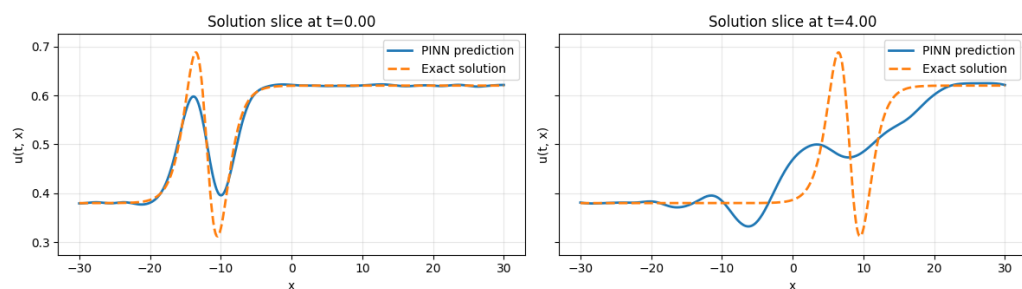


Рисунок 3 – 3000 эпоха

По мере оптимизации сеть начинает улавливать глобальную геометрию решения. Проявляется характерный профиль: пик и следующий за ним провал. Однако на этом этапе наблюдаются фазовые сдвиги и ошибки амплитуды. Модель успешно выучила начальное условие при $t = 0$, но физическая невязка

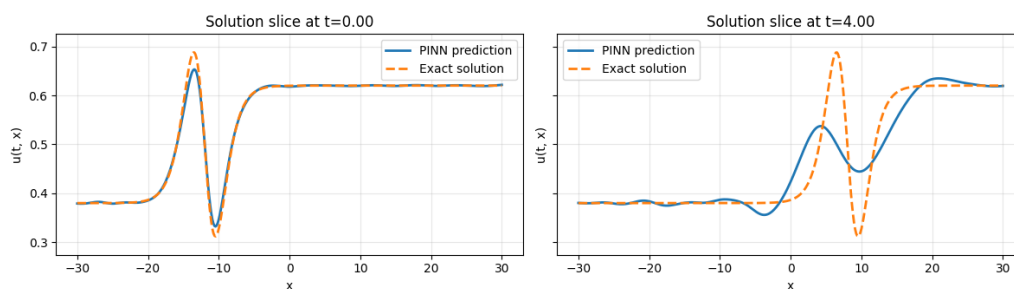


Рисунок 4 – 7000 эпоха

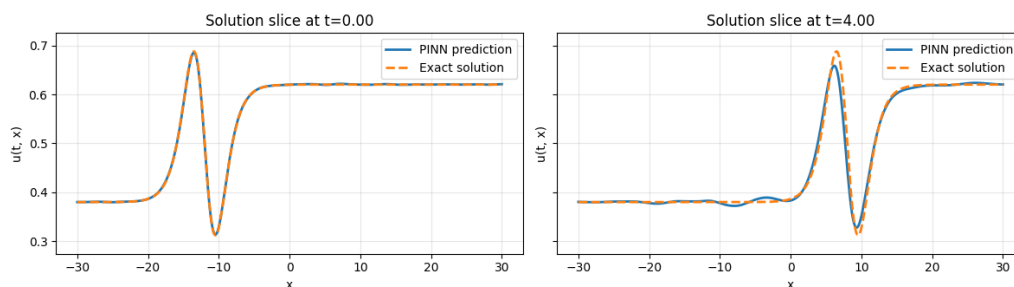


Рисунок 5 – 13000 эпоха

(уравнение) еще не до конца сбалансирована. Конвективный член uu_x и баланс диффузии сопротивляются корректному переносу структуры во времени, из-за чего на срезе $t = 4$ предсказание может отставать или опережать аналитическую бегущую волну.

Третья фаза - асимптотическая сходимость (Итоговая аппроксимация, рисунок 6):**

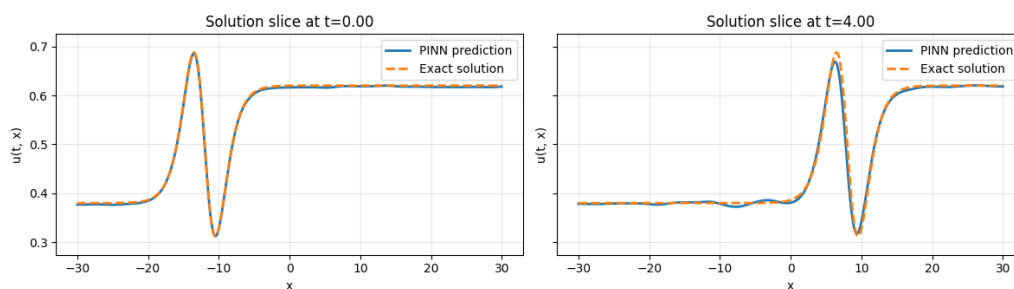


Рисунок 6 – 14000 эпоха

На финальных стадиях градиенты всех компонент функции потерь приходят к равновесию. Нейросеть идеально восстанавливает решение. Синяя кривая практически полностью сливается с эталонной оранжевой кривой на обоих временных срезах.

В результате обучения была достигнута хорошая геометрическая точность, воспроизведен нелинейный профиль решения ($\tanh / \tanh^3$) с сохранением

масштаба амплитуды.

2.4 Анализ ошибки

Как следует из рисунок 7, динамика сходимости различных компонент функции потерь существенно отличается. На начальном этапе обучения модель демонстрирует более точную аппроксимацию начального условия (1) по сравнению с граничными условиями Неймана (3). Это проявляется в более быстром убывании соответствующей компоненты функции потерь.

Однако по мере продолжения обучения наблюдается изменение характера сходимости: условия Неймана (3) демонстрируют более устойчивое поведение и меньшую чувствительность к итерационным колебаниям. В то же время аппроксимация начального условия (1) становится менее стабильной, что выражается в появлении значительных и частых осцилляций ошибки. Подобное поведение может свидетельствовать о переобучении модели по данной компоненте.

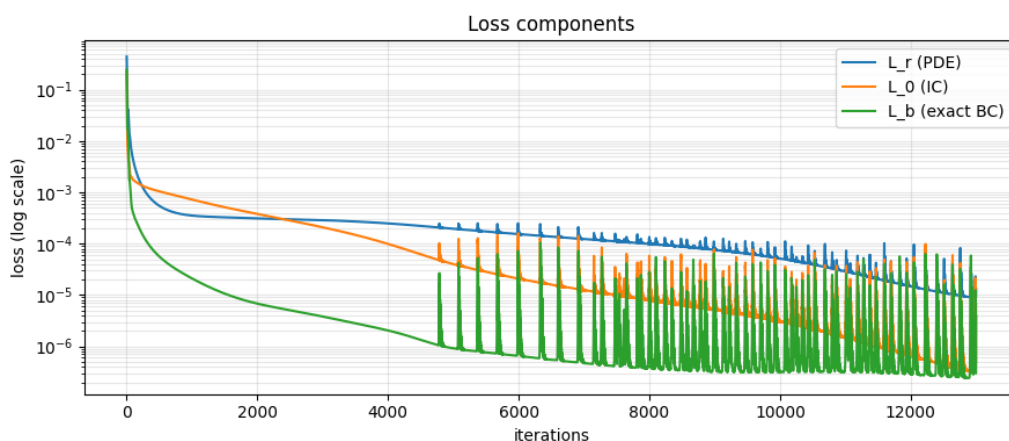


Рисунок 7 – Динамика компонент функции потерь, соответствующих различным условиям задачи

Суммарная функция потерь (рисунок 8) в целом демонстрирует хорошую сходимость: значение ошибки монотонно убывает на протяжении большей части процесса обучения и достигает порядка 10^{-5} , что можно считать качественным результатом аппроксимации. Тем не менее, на заключительном этапе обучения наблюдается рост числа осцилляций, что обусловлено увеличением вклада ошибки, связанной с граничным условием (1). Это указывает на ухудшение обобщающей способности модели в данной области

и возможную необходимость дополнительной регуляризации или балансировки весов в функции потерь.

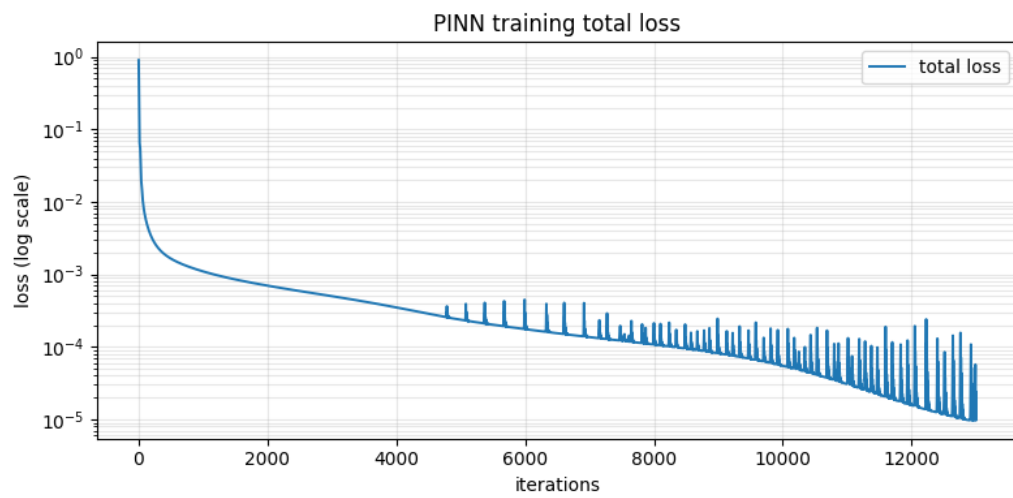


Рисунок 8 – Динамика суммарной функции потерь

Глава 3 Глава 3: Анализ 2 этапа и сравнение с 1

Модификация постановки задачи и варианты архитектур PINN

В качестве одного из способов сокращения времени обучения и повышения точности аппроксимации была рассмотрена модификация исходной постановки задачи, основанная на введении дополнительной неизвестной. Для этого использовалась замена

$$v = \frac{\partial^2 u}{\partial x^2}.$$

Такая подстановка позволяет понизить порядок старшей пространственной производной в исходном уравнении: вместо производной четвертого порядка рассматривается система, содержащая производные не выше второго порядка. С вычислительной точки зрения это особенно важно для PINN-подхода, поскольку вычисление высоких производных с помощью автоматического дифференцирования, как правило, приводит к усложнению оптимизации, росту чувствительности к выбору гиперпараметров и ухудшению устойчивости обучения.

В результате исходная задача преобразуется в систему уравнений относительно двух неизвестных функций $(u(t,x))$ и $(v(t,x))$, дополненную условием согласования

$$v - \frac{\partial^2 u}{\partial x^2} = 0.$$

Тем самым дополнительная переменная (v) не является независимой в физическом смысле, а вводится как вспомогательная функция, обеспечивающая снижение дифференциального порядка задачи. При этом в функционал потерь PINN включается соответствующая невязка, гарантирующая выполнение связи между (u) и (v) .

Для реализации данной идеи были исследованы три варианта архитектуры нейронной сети.

1. Одна нейронная сеть с двумя выходами

В первом подходе использовалась одна нейронная сеть, которая по входным переменным $((t,x))$ одновременно аппроксимирует обе неизвестные:

$$(t, x) \mapsto (u_\theta(t, x), v_\theta(t, x)).$$

С математической точки зрения это означает построение единого

параметрического отображения

$$\mathcal{N}_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}^2,$$

где первый выходной нейрон соответствует приближению функции (u), а второй — функции (v).

В данной архитектуре входные переменные предварительно преобразуются с помощью фурье-признаков, что позволяет лучше представлять многомасштабные и осциллирующие зависимости. После этого объединенный вектор признаков передается через несколько скрытых полносвязных слоев, а на последнем слое формируется двумерный выход. Таким образом, обе искомые функции обучаются совместно и используют общее внутреннее представление признаков.

Преимущество такого подхода состоит в компактности модели и в возможности совместного обучения взаимосвязанных величин. Однако общий набор параметров для двух неизвестных может приводить к конкуренции между задачами аппроксимации (u) и (v), особенно если соответствующие компоненты функции потерь имеют различную сложность или различный масштаб.

Две независимые нейронные сети с одним выходом

Во втором подходе каждая неизвестная аппроксимируется собственной нейронной сетью:

$$u_\theta(t, x) = \mathcal{N}^{(u)} * \theta_u(t, x), \quad v * \theta(t, x) = \mathcal{N}_{\theta_v}^{(v)}(t, x).$$

Здесь каждая модель имеет один выходной нейрон и отвечает только за одну функцию. Архитектурно обе сети могут быть одинаковыми, однако их параметры не разделяются.

Математически этот вариант соответствует отдельной параметризации компонент системы. Такое разделение повышает гибкость модели, поскольку каждая сеть может адаптироваться к особенностям своей функции без необходимости согласовывать внутреннее представление с другой неизвестной. Это особенно полезно в случаях, когда функции (u) и (v) имеют различную гладкость, различный масштаб значений или разный характер пространственно-временной зависимости.

С другой стороны, полное разделение сетей увеличивает общее число параметров и может ухудшать использование общей информации о структуре

решения, которая в многовыходной сети извлекается автоматически за счет совместного скрытого представления.

Сеть с общим стволом и отдельными выходными ветвями

Третий вариант представляет собой компромисс между двумя предыдущими подходами. Используется единая общая часть сети, формирующая скрытое представление входных данных, после чего вычисления разделяются на две независимые выходные ветви:

$$h(t, x) = \mathcal{T} * \theta(t, x),$$

$$u * \theta(t, x) = \mathcal{H}^{(u)} * \theta_u(h), \quad v * \theta(t, x) = \mathcal{H}^{(v)} * \theta_v(h).$$

Здесь $\mathcal{T} * \theta$ — общий блок (shared trunk), а $\mathcal{H}^{(u)} * \theta_u$ и $\mathcal{H}^{(v)} * \theta_v$ — специализированные выходные головы для (u) и (v) соответственно.

Такой подход позволяет на ранних этапах обработки использовать общую информацию о задаче, поскольку обе неизвестные зависят от одних и тех же переменных ((t,x)) и описывают одно и то же решение с разных сторон. Одновременно отдельные финальные ветви дают модели дополнительную свободу для учета индивидуальных особенностей каждой функции. В контексте PINN это часто рассматривается как более сбалансированная архитектура: общие слои извлекают универсальные признаки решения, а специализированные ветви адаптируются к конкретным ограничениям, связанным с аппроксимацией (u) и (v).

Сравнение качества рассмотренных архитектур

Для оценки эффективности указанных подходов были сопоставлены минимальные и финальные значения суммарной функции потерь. Результаты обучения представлены ниже:

$$\begin{aligned} \text{stage1_start : } & \text{best_total} = 9.691403 \cdot 10^{-6}, \quad \text{final_total} = 2.382773 \cdot 10^{-5}, \\ \text{two_output : } & \text{best_total} = 1.446138 \cdot 10^{-4}, \quad \text{final_total} = 1.446138 \cdot 10^{-4}, \\ \text{two_models : } & \text{best_total} = 1.425359 \cdot 10^{-4}, \quad \text{final_total} = 1.436357 \cdot 10^{-4}, \\ \text{shared_trunk : } & \text{best_total} = 1.526569 \cdot 10^{-4}, \quad \text{final_total} = 1.536401 \cdot 10^{-4}. \end{aligned}$$

Из приведенных данных следует, что наилучший результат показала исходная постановка без введения дополнительной переменной (v):

минимальное значение функции потерь составило порядка 10^{-5} , что примерно на порядок лучше результатов всех трех модифицированных архитектур. Среди моделей с заменой $v = \partial^2 u / \partial x^2$ наилучшее значение достигнуто при использовании двух независимых сетей с одним выходом, однако различия между тремя новыми архитектурами остаются сравнительно небольшими.

Таким образом, введение дополнительной переменной и переход к системе меньшего порядка само по себе не привели к улучшению точности. Несмотря на теоретическое упрощение дифференциальной структуры задачи, на практике добавление новой неизвестной и дополнительного условия согласования увеличило сложность оптимизационной задачи. Вероятно, выигрыш от уменьшения порядка производной был компенсирован необходимостью одновременно обеспечивать точную аппроксимацию двух взаимосвязанных функций и выполнение ограничения

$$v - \frac{\partial^2 u}{\partial x^2} = 0.$$

Следовательно, для рассматриваемой задачи более перспективным может быть не усложнение постановки путем введения вспомогательной переменной, а дальнейшая настройка исходной архитектуры, выбор весов отдельных слагаемых функции потерь, а также применение методов стабилизации обучения.

1. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019.
2. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. *SIAM Journal on Scientific Computing*, 2021.
3. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 2022.
4. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 2023.
5. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. *NeurIPS*, 2020.
6. On the Spectral Bias of Neural Networks. *ICML*, 2019.
7. Diffusion-Induced Chaos in Reaction Systems. *Progress of Theoretical Physics Supplement*, 1978.
8. Nonlinear analysis of hydrodynamic instability in laminar flames – I. Derivation of basic equations. *Acta Astronautica*, 1977.
9. On Flame Propagation Under Conditions of Stoichiometry. *SIAM Journal on Applied Mathematics*, 1980.
10. The Well-Posedness of the Kuramoto–Sivashinsky Equation. *SIAM Journal on Mathematical Analysis*, 1986.
11. The Kuramoto–Sivashinsky Equation (lecture notes), 2006.
12. Numerical solutions of the generalized Kuramoto–Sivashinsky equation by Chebyshev spectral collocation methods. *Computers & Mathematics with Applications*, 2008.
13. Approximate Solution of Kuramoto–Sivashinsky Equation Using Reduced Differential Transform Method, 2014.
14. Physics-Informed Neural Networks with Hard Constraints for Inverse Design. *SIAM Journal on Scientific Computing*, 2021.

15. FP64 is All You Need: Rethinking Failure Modes in Physics-Informed Neural Networks. *NeurIPS poster*, 2025.